TECHNICAL TRAINING

Kubernetes Architecture

How the control plane, network, storage and security models turn declared intent into running, reachable and contained workloads.

Vivekanand Pujar | @VivekanandPujar

Consultant-led workshop · Platform engineers, SREs, architects and engineering leadership

Four sessions · Architecture, networking, storage, security and RBAC

SESSION MAP

Agenda

01 Kubernetes architecture

Control plane, nodes and the reconciliation loop that turns declared intent into running Pods.

02 The networking model

Flat Pod networking, Services, Gateway API, CNI plugins and NetworkPolicy.

03 The storage model

Claims, volumes, classes and CSI, and how state survives Pods that do not.

04 Security and RBAC

Authentication, authorisation, admission, and the layers that contain a compromised Pod.

Each session follows one request, one packet, one byte or one identity through the system, and names where it can fail.

Vivekanand Pujar | @VivekanandPujar

THE ARGUMENT

What this course is trying to establish

> Kubernetes does not run your application. It stores what you asked for and runs controllers that keep trying to make the cluster match it. Every design decision is a decision about what those controllers can see, and what they are allowed to change.

Intent is data, not a command

A manifest is a record in etcd, not a script. Nothing happens because you applied it; something happens because a controller noticed the difference and acted.

The plug decides the behaviour

Kubernetes defines the network, storage and identity contracts. The CNI, CSI driver and identity provider you choose decide how those contracts behave under failure.

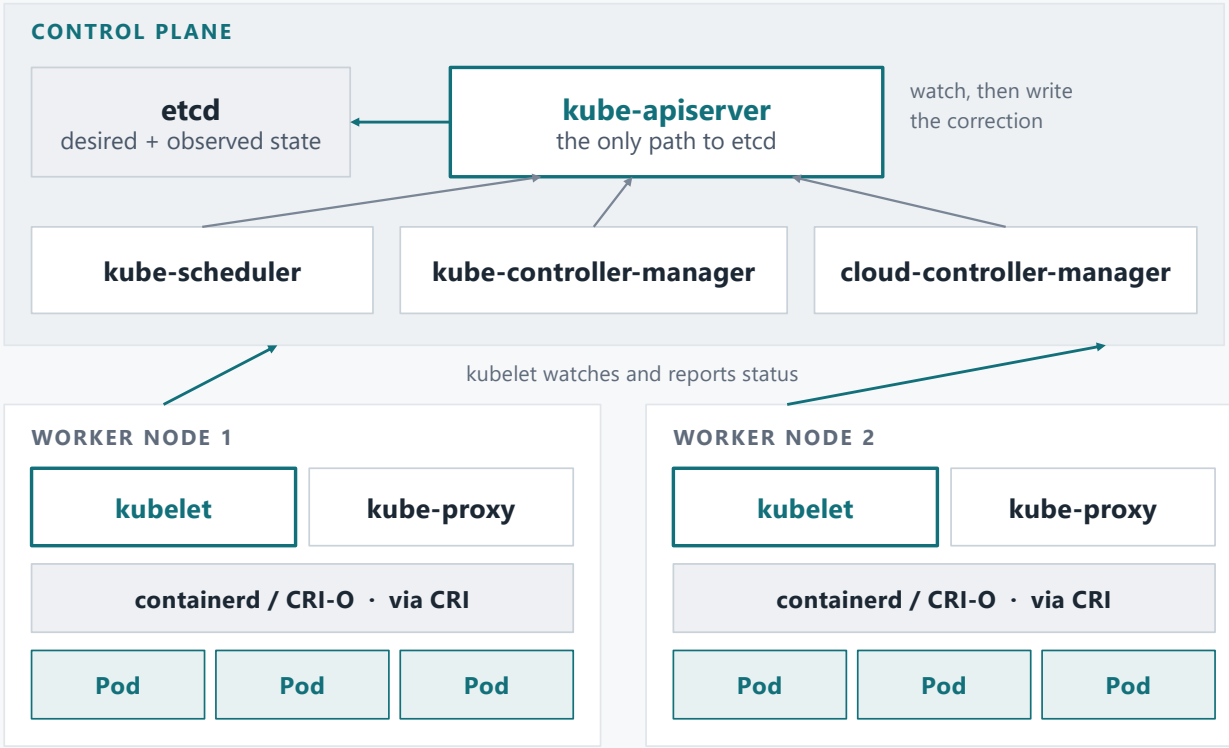
Access is additive and silent

RBAC has no deny rule. A permission granted once, to one service account, is available to every Pod that can use that account, until someone removes it.

Vivekanand Pujar | @VivekanandPujar

SESSION 01

The control plane stores intent; every other component reconciles toward it



WHAT EACH PART OWNS

- **kube-apiserver** authenticates, authorises, admits and persists every object. It is stateless and scales horizontally.
- **etcd** is the only stateful component: a Raft-replicated store run as three or five members.
- **Controllers** compare desired with observed state and write corrections. The scheduler is one: it sets spec.nodeName.
- **kubelet** makes the Pods bound to its node match their specs, through the container runtime interface.

WHAT IT IMPLIES

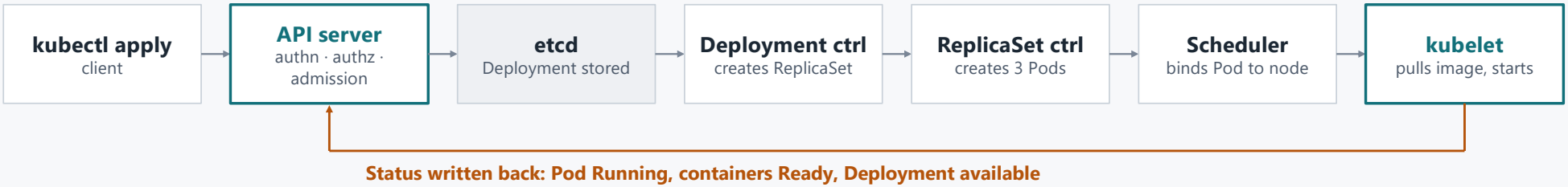
No component calls another directly; all coordination runs through watches on the API server. Losing the control plane freezes change, not the workloads already running.

Vivekanand Pujar | @VivekanandPujar

SESSION 01

Every change is a write to the API server, and everything else reacts to it

ONE DEPLOYMENT, FROM kubectl apply TO A RUNNING POD



After etcd, every step is a separate component. None calls the next; each watches the API server for objects it owns and writes its result back.

Level-triggered, not edge-triggered

Controllers act on the current difference, not the event that caused it. A missed event is repaired on the next resync, so a controller that restarts loses nothing.

Applied is not deployed

No transaction spans the steps. The Deployment is accepted while its Pods stay Pending on quota, taints or image pulls. kubectl apply returns once etcd has the object.

The API server is the scaling limit

Every controller, kubelet and operator holds watches on it. Unbounded LIST calls from chatty operators degrade large clusters sooner than Pod count does.

Wait on the outcome with kubectl rollout status, readiness gates or a GitOps health check; never on the exit code of apply.

Vivekanand Pujar | @VivekanandPujar

SESSION 01

Each component fails differently, and the cluster degrades in a known order

COMPONENT	WHAT IT OWNS	WHEN IT FAILS	WHAT HOLDS
etcd	Every object and its version; the cluster's only durable state	Quorum lost: writes fail and the API stops serving	Odd member count, dedicated disks, tested snapshot restore
kube-apiserver	Authn, authz, admission and validation for every read and write	No change possible; running Pods keep serving traffic	Several replicas behind a load balancer; API priority and fairness
kube-scheduler	Placement of new Pods against resources, affinity and taints	New Pods stay Pending; existing Pods are unaffected	Leader election across replicas; watch Pending age
controller-manager	Replica counts, node lifecycle, endpoints, jobs, garbage collection	Failed Pods are not replaced; scale events do nothing	Leader election; alert on work-queue depth
kubelet	Pod lifecycle, probes, volume mounts on one node	Node NotReady; Pods evicted after the toleration window	Reserve system resources; node problem detector
CoreDNS	Service and Pod name resolution inside the cluster	Every service-to-service call fails despite healthy Pods	Replicas across zones, NodeLocal DNSCache, ndots tuning

The data plane outlives the control plane: an API outage stops deployments and self-healing, but traffic to running Pods continues.

SESSION 01

Workload objects are contracts, not deployment scripts

01 Pod

What runs together?

Containers sharing one network namespace and volumes. Scheduled as a unit; never durable.

> *A bare Pod on a failed node is gone; nothing recreates it.*

02 Deployment

Which version, how many, how fast?

Manages ReplicaSets for stateless replicas, within maxSurge and maxUnavailable.

> *Without a readiness probe, a rollout counts broken Pods as available.*

03 StatefulSet

Which replica is which?

Stable names, ordered start-up and a volume per replica, for databases and quorum systems.

> *Treated as a database operator; it replicates no data itself.*

04 DaemonSet

What must run on every node?

One Pod per matching node: log shippers, CNI agents, CSI node plugins, monitoring.

> *One heavy agent's requests, multiplied by every node in the fleet.*

05 Job and CronJob

What must finish?

Run to completion with retries, parallelism and a backoff limit; CronJob adds a schedule.

> *A non-idempotent job reruns after node loss and charges twice.*

06 HorizontalPodAutoscaler

How many replicas right now?

Adjusts replica count from CPU, memory or custom metrics, relative to requests.

> *No resource requests set, so the utilisation target means nothing.*

Controllers own their Pods through ownerReferences; delete the owner and garbage collection removes everything it created.

Vivekanand Pujar | @VivekanandPujar

SESSION 02

The network model rests on four promises, not on one implementation

Container to container

Containers in one Pod share a network namespace and reach each other on localhost.

IMPLEMENTED BY

The Pod sandbox, which holds the shared network namespace.

WHAT CATCHES TEAMS OUT

Two containers in a Pod cannot bind the same port.

Pod to Pod

Every Pod gets its own IP and can reach every other Pod without NAT, on any node.

IMPLEMENTED BY

The CNI plugin: routes, overlays or native VPC addressing.

WHAT CATCHES TEAMS OUT

The Pod CIDR is sized once; exhausting it stops scaling.

Pod to Service

A Service gives a changing set of Pods one stable virtual IP and DNS name.

IMPLEMENTED BY

kube-proxy rules (iptables, IPVS, nftables) or eBPF.

WHAT CATCHES TEAMS OUT

Clients that cache resolved Pod IPs outlive the Pods.

Outside to Service

NodePort, LoadBalancer and Gateway or Ingress resources admit external traffic.

IMPLEMENTED BY

A cloud load balancer plus a Gateway or Ingress controller.

WHAT CATCHES TEAMS OUT

Source IPs are lost to SNAT unless externalTrafficPolicy is Local.

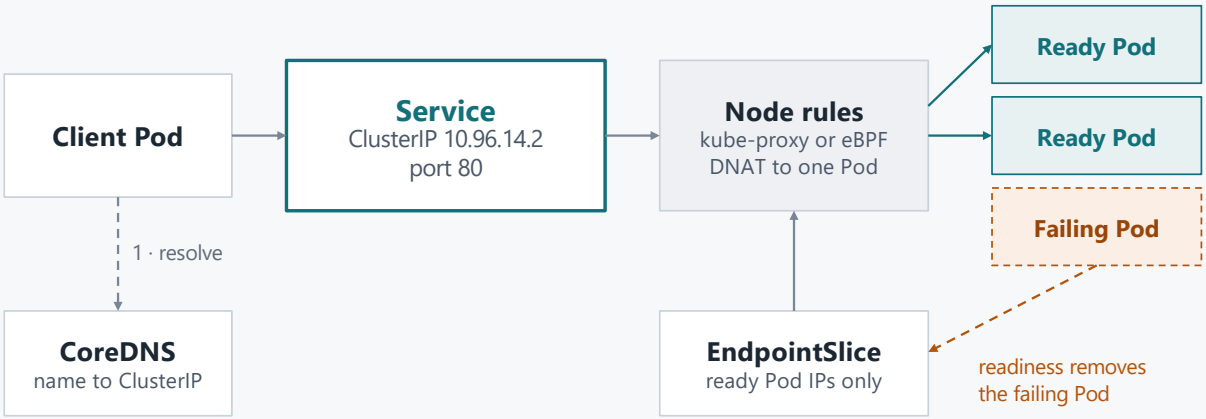
Kubernetes specifies the promises; the CNI plugin, kube-proxy mode and load-balancer controller you install decide how each one is kept.

Vivekanand Pujar | @VivekanandPujar

SESSION 02

A Service is a stable name in front of a changing set of Pods

ONE CALL TO checkout.shop.svc.cluster.local



No process called Service exists. It is an API record; the EndpointSlice controller tracks which Pods are ready, and every node rewrites packets addressed to the virtual IP toward one of them.

kube-proxy's nftables mode is GA since v1.33; Cilium and Calico eBPF can replace kube-proxy entirely.

FIVE SERVICE TYPES

- **ClusterIP** Virtual IP reachable inside the cluster only. The default.
- **NodePort** Opens the same port on every node; the building block for external load balancers.
- **LoadBalancer** Asks the cloud controller for an external load balancer in front of NodePorts.
- **Headless** clusterIP: None. DNS returns Pod IPs directly; used by StatefulSets and client-side balancing.
- **ExternalName** A DNS CNAME to something outside the cluster; no proxying.

WHAT HOLDS

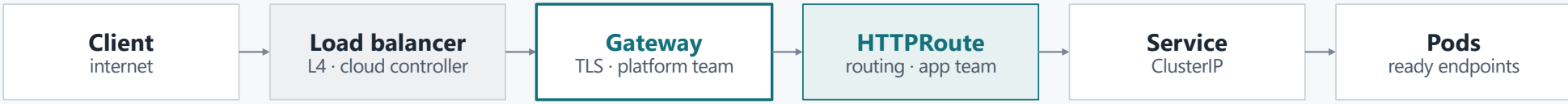
Readiness probes that report whether this Pod can serve, and client retries that tolerate endpoint propagation delay.

Vivekanand Pujar | @VivekanandPujar

SESSION 02

Traffic from outside crosses three layers, and the Ingress layer is being replaced

NORTH-SOUTH PATH FOR ONE HTTPS REQUEST



Gateway API splits one Ingress object into roles: GatewayClass for the infrastructure provider, Gateway for the platform team, Routes for application teams.

	INGRESS	GATEWAY API	SERVICE MESH
Status	Stable, feature-frozen; no new capabilities planned	Standard GA channel; the community's successor to Ingress	East-west layer; Istio, Linkerd and Cilium implement it
Expressiveness	Host and path; everything else in vendor annotations	Headers, weights, mirroring, timeouts, TLS and gRPC as typed fields	Retries, mTLS, circuit breaking and identity between services
Ownership	One object mixes platform and application concerns	Role-oriented resources, each bound by RBAC to its owner	Policy per workload; the mesh control plane is new infrastructure
Watch for	ingress-nginx retired in March 2026: no further security fixes	Implementations differ in the extended features they support	Sidecar or ambient mode changes latency and failure behaviour

Source: Kubernetes SIG Network and Security Response Committee, “Ingress NGINX Retirement”, kubernetes.io, November 2025. Migration tooling: ingress2gateway 1.0.

Vivekanand Pujar | @VivekanandPujar

SESSION 02

The CNI plugin decides how packets move, and what policy can enforce

PLUGIN	DATA PATH	WHAT IT BUYS	WHAT IT COSTS
Cilium	**eBPF**	kube-proxy replacement, L3 to L7 policy, Hubble flow visibility	Kernel version floor; eBPF debugging is a new skill for the team
Calico	**BGP, IPIP, VXLAN or eBPF**	Mature NetworkPolicy plus global policy; routes without overlay where BGP is available	BGP peering with the physical network needs a network team
Flannel	**VXLAN overlay**	The simplest install; enough for labs and small clusters	No NetworkPolicy on its own; pair it with Calico (Canal)
AWS VPC CNI	**Native VPC IPs**	Pods addressable from the VPC; no encapsulation overhead	Pod density bounded by ENI and IP limits per instance type
Azure CNI Overlay	**Overlay or native VNet**	Large Pod CIDRs without consuming VNet address space	Pod IPs are not routable from the VNet; inbound traffic needs a load balancer
GKE Dataplane V2	**Cilium-based eBPF**	Built-in policy enforcement and logging, managed by Google	Behaviour and versions follow the provider, not upstream

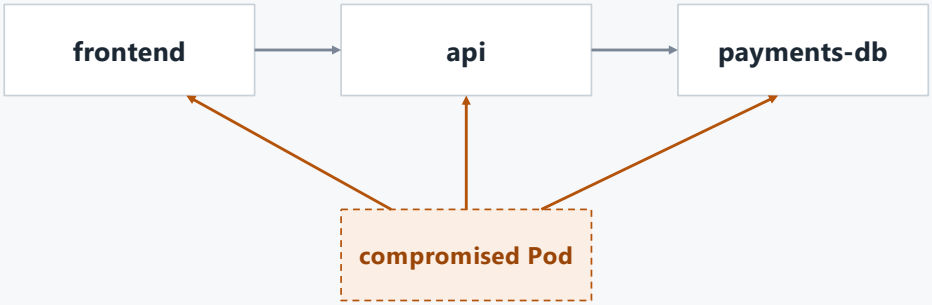
Choose the CNI for its policy model and address plan first. Throughput differences are rarely the constraint; a Pod CIDR that runs out is.

Vivekanand Pujar | @VivekanandPujar

SESSION 02

Every Pod can reach every Pod until a NetworkPolicy says otherwise

THE DEFAULT · NO POLICY IN THE NAMESPACE

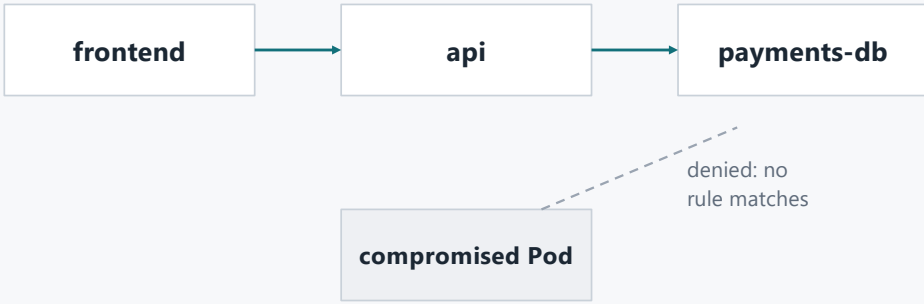


Reaches the database directly; nothing records or refuses it.

NetworkPolicy is additive and applies only to Pods it selects. A namespace with no policy is fully open, including to Pods in every other namespace.

Policy is enforced by the CNI. On a plugin without policy support the objects are accepted and silently ignored. Namespaces and labels, not IPs, are the unit of rule.

THE POSTURE THAT HOLDS · DEFAULT-DENY, THEN ALLOW



Only frontend to api and api to payments-db are allowed.

Select every Pod with an empty podSelector and deny both directions, then add one policy per permitted flow. Remember DNS: egress to CoreDNS on port 53 must be allowed.

```
podSelector: {}
policyTypes: [Ingress, Egress]
```

Vivekanand Pujar | @VivekanandPujar

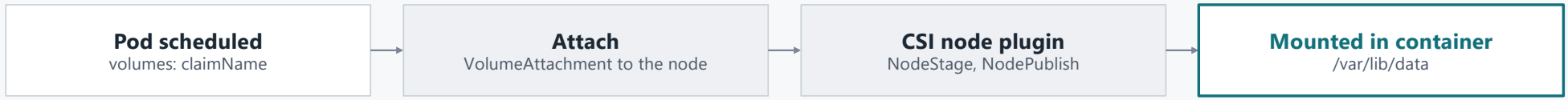
SESSION 03

Storage is split into claim, volume and class, so an application never names a disk

PROVISIONING · CONTROL PLANE, ONCE PER CLAIM



CONSUMPTION · EVERY TIME A POD USING THE CLAIM STARTS



The application names a need

A claim asks for size, access mode and class. Moving from one cloud to another changes the StorageClass, not the manifests of every team.

Drivers live outside Kubernetes

In-tree volume plugins have migrated to CSI. Snapshots, expansion and topology are driver features, so the driver's version decides what works.

Binding is exclusive

One PV binds to one PVC. Deleting the claim triggers the reclaim policy, and under Delete the backend disk goes with it.

Vivekanand Pujar | @VivekanandPujar

SESSION 03

Volume types differ in lifetime, and the lifetime decides what data is lost

VOLUME	LIVES AS LONG AS	RIGHT FOR	WHAT GOES WRONG
emptyDir	**the Pod**	Scratch space, caches, files shared between containers	Treated as durable; a reschedule or eviction wipes it
configMap, secret, projected	**the source object**	Configuration, certificates and service-account tokens as files	Mounted with subPath, so updates never reach the container
hostPath	**the node**	Node agents that must read host logs or sockets	A Pod gains write access to the node; blocked by restricted policy
Generic ephemeral volume	**the Pod, on real storage**	Large scratch data that needs a CSI driver's performance	Provisioning latency added to every Pod start
PersistentVolume, block	**the claim, beyond any Pod**	Databases and queues; ReadWriteOnce, pinned to one zone	The Pod cannot follow its disk to a node in another zone
PersistentVolume, file	**the claim, beyond any Pod**	Shared content across replicas; ReadWriteMany over NFS or SMB	Concurrent writers with no locking protocol corrupt data

Local persistent volumes offer disk speed with node affinity: when the node is lost, so is the data, unless the application replicates it.

SESSION 03

Three storage settings decide what happens when a node, a Pod or a claim disappears

Access mode

ReadWriteOnce: one node.
ReadWriteOncePod: one Pod.
ReadOnlyMany and **ReadWriteMany**: many nodes, if the backend supports it.

THE SAFE CHOICE

ReadWriteOncePod for single-writer databases; the kubelet enforces it.

WHAT GOES WRONG

Two replicas mount one RWO disk on the same node and both write to it.

Reclaim policy

Delete removes the backend disk with the claim. **Retain** keeps the PV and its data for an administrator to recover.

THE SAFE CHOICE

Retain for any system of record, with snapshots taken by VolumeSnapshot.

WHAT GOES WRONG

A namespace is deleted during clean-up and the production database goes with it.

Volume binding mode

Immediate provisions when the claim is created. **WaitForFirstConsumer** waits until a Pod using it is scheduled.

THE SAFE CHOICE

WaitForFirstConsumer for zonal block storage, so the disk is created in the Pod's zone.

WHAT GOES WRONG

The disk lands in zone a, the only free node is in zone b, and the Pod stays Pending.

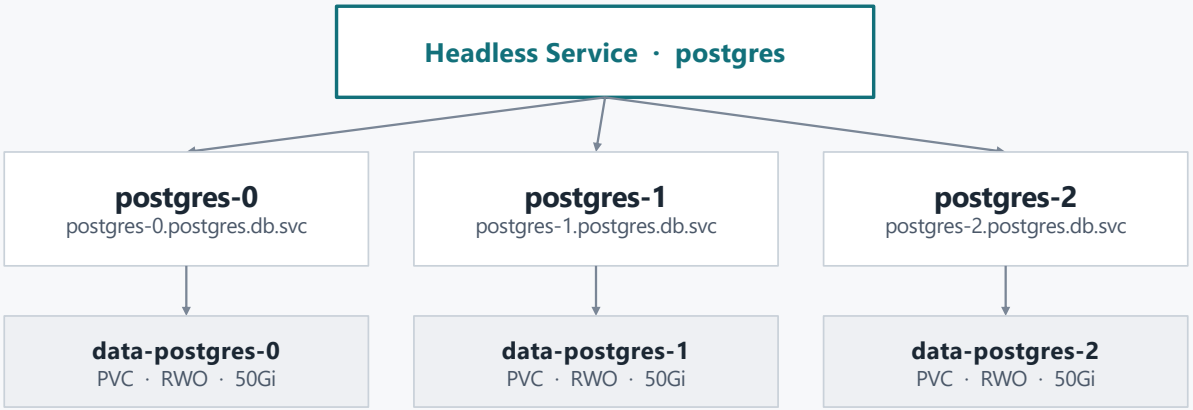
Set these in the StorageClass, where the platform team owns them, rather than leaving each team to choose per claim.

Vivekanand Pujar | @VivekanandPujar

SESSION 03

StatefulSets give each replica an identity and its own volume, and nothing more

STATEFULSET POSTGRES · 3 REPLICAS · ONE VOLUME CLAIM TEMPLATE



Start-up is ordered 0, 1, 2 and scale-down runs in reverse. A rescheduled postgres-1 comes back with the same name and re-attaches data-postgres-1.

Replica 0 is not a primary. Leader election, replication and failover belong to the database or its operator (CloudNativePG, Strimzi, Vitess); the StatefulSet only keeps names and disks stable.

WHAT IT GUARANTEES

- A stable network identity per replica.
- A volume that follows the replica across nodes.
- Ordered, one-at-a-time rollout and scaling.

WHAT IT DOES NOT

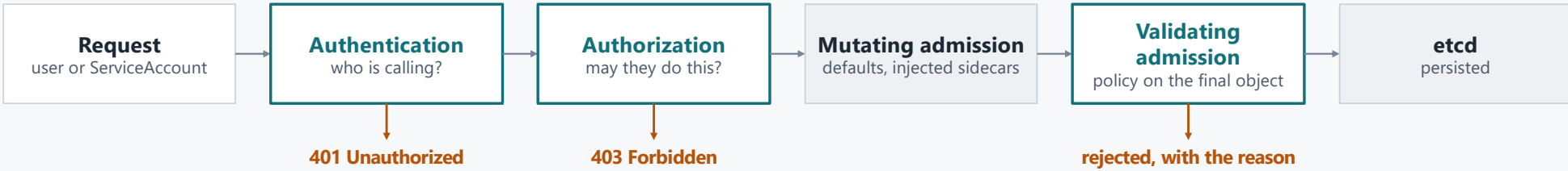
- Replicate data between replicas.
- Delete the PVCs when scaled down or deleted, unless persistentVolumeClaimRetentionPolicy says so.
- Back anything up; that is VolumeSnapshot plus an off-cluster copy.
- Move a zonal disk: a replica whose zone fails stays Pending.

Vivekanand Pujar | @VivekanandPujar

SESSION 04

Every request crosses three gates before it reaches etcd

ONE REQUEST: CREATE A POD IN NAMESPACE payments



Authentication

Kubernetes stores no user accounts. People arrive with an OIDC token or a client certificate; workloads use ServiceAccount tokens, which are now short-lived and audience-bound.

PRACTICE

OIDC through the company identity provider; no long-lived client certificates.

Authorization

Authorizers run in order (Node, RBAC, webhooks) and the first to allow wins. RBAC can only grant access; it has no deny rule.

PRACTICE

Groups from the identity provider bound to namespaced Roles.

Admission

The last point where policy sees the whole object. Pod Security Admission and ValidatingAdmissionPolicy run inside the API server; webhooks add Kyverno or Gatekeeper.

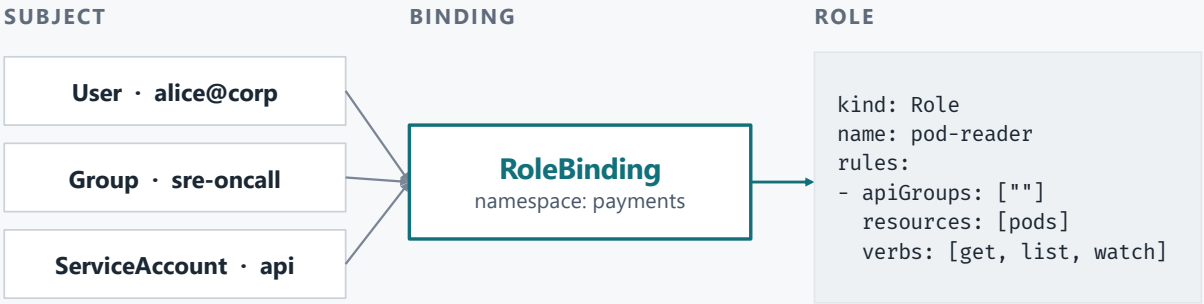
PRACTICE

CEL policies in the API server first; a webhook only where they cannot express the rule.

Vivekanand Pujar | @VivekanandPujar

SESSION 04

RBAC binds subjects to verbs on resources, and grants only ever add up



COMBINATION	WHAT IT GRANTS
Role + RoleBinding	The rules, in one namespace only
ClusterRole + RoleBinding	A shared definition, applied in one namespace
ClusterRole + ClusterRoleBinding	The rules in every namespace, plus nodes, PVs and CRDs

Built-in ClusterRoles view, edit and admin are aggregated: installing a CRD can widen them automatically. cluster-admin should be bound to a break-glass group, never to people.

HOW TO READ A GRANT

- Access is the union of every binding that names the subject, including its groups.
- There is no deny rule; to remove access, find and delete the grant.
- Verbs map to HTTP: get, list, watch, create, update, patch, delete.

WHAT HOLDS

- Bind groups from the identity provider, not individual users.
- One ServiceAccount per workload, with automountServiceAccountToken off unless the app calls the API.
- Audit with kubectl auth can-i --list and review bindings in CI.

Vivekanand Pujar | @VivekanandPujar

SESSION 04

Six grants that quietly come close to cluster-admin

THE GRANT	WHY IT ESCALATES	WHAT TO GRANT INSTEAD
create on pods	A Pod can mount any ServiceAccount's token or a hostPath in the namespace, and run as that identity	Create through Deployments owned by a CI identity; Pod Security at restricted
get or list on secrets	list returns every Secret's contents, including tokens and database credentials	resourceNames for the named Secrets a workload needs, or an external secrets store
escalate or bind on roles	Lets a subject write or bind a Role with more rights than it holds itself	Only the platform's GitOps identity manages RBAC objects
impersonate	Acts as any user, group or ServiceAccount, including system:masters	Limit to named subjects with resourceNames, for support tooling only
nodes/proxy, pods/exec	Direct access to the kubelet API or a shell inside running workloads	Time-bound access through a just-in-time grant with audit
Wildcards: * on * in *	Silently includes every resource added later, CRDs included	Explicit resources and verbs, reviewed as code

Test before binding: kubectl auth can-i create pods --as=system:serviceaccount:payments:api -n payments.

Vivekanand Pujar | @VivekanandPujar

SESSION 04

Pod security is layered; each layer covers what the one before it misses

01 Pod Security Admission

What may a Pod ask for?

Namespace labels enforce baseline or restricted: no privileged, host-namespace or root Pods.

> *Namespaces left at privileged because one DaemonSet needed it.*

02 securityContext

How does the container run?

runAsNonRoot, readOnlyRootFilesystem, drop ALL capabilities, seccomp RuntimeDefault.

> *Images that only start as root, so the policy is waived for them.*

03 ServiceAccount tokens

What can the Pod call?

Projected, short-lived tokens bound to one audience; automount off by default.

> *The default ServiceAccount carrying a RoleBinding someone added once.*

04 NetworkPolicy

Who can the Pod reach?

Default-deny per namespace, with explicit ingress and egress, DNS included.

> *Policies written for ingress only, so any Pod can exfiltrate data.*

05 Secrets

Where do credentials live?

Encryption at rest with a KMS provider, or an external store synced by an operator.

> *Base64 in Git, read by anyone with list on secrets.*

06 Supply chain and audit

What is allowed to run, and who did what?

Signed images verified at admission; API audit logs shipped off the cluster.

> *A compromise discovered weeks later, with no log of the first request.*

PodSecurityPolicy was removed in v1.25. Pod Security Admission is built in; Kyverno or Gatekeeper cover rules its three profiles cannot express.

Vivekanand Pujar | @VivekanandPujar

CLOSING

Ten questions for the next cluster review

ARCHITECTURE

01 When etcd was last restored from a snapshot, how long did it take?

02 Which controllers or operators issue unbounded LIST calls against the API server?

03 Which workloads have no readiness probe, so a rollout cannot tell broken from ready?

NETWORKING

04 Which namespaces have no default-deny policy, in both directions?

05 What replaces ingress-nginx, and which annotations have no Gateway API equivalent?

STORAGE

06 Which claims hold a system of record under a Delete reclaim policy?

07 When a zone fails, which StatefulSet replicas stay Pending, and for how long?

SECURITY AND RBAC

08 Who can create Pods, read Secrets or impersonate, and does anyone review it?

09 Which ServiceAccounts mount a token they never use?

10 Could the first API request of an intrusion be found in the audit log today?

Bring the manifests for one service you own. We will trace its request, packet, volume and identity through the cluster and find the weakest step.

Recommended reading: Kubernetes documentation, Concepts; Burns et al., Kubernetes: Up and Running.

Vivekanand Pujar | @VivekanandPujar